

EcoHub: IoT Device Manager & Analytics Engine

The Project Proposal

Project Name: "EcoHub: IoT Device Manager & Analytics Engine"

Objective: Build a central control system that simulates and manages a network of smart devices (Sensors and Actuators). The system must handle asynchronous data streams from devices, process data pipelines to trigger automation rules, and safely archive history to disk.

Core Requirements by Module

1. The Device Layer (OOP & Abstraction)

- **Abstract Base Class:** Create `SmartDevice` with methods like `connect()`, `send_update()`, and an abstract method `execute_command()`.
- Every device must have these properties:
 - `device_id`: the unique id of the device
 - `name`: the name of the device used to identify it in the ecosystem
 - `location`: the location where this device is installed
 - `device_type`: for the abstract class would be "GENERIC"
- **Inheritance:** Implement *concrete* classes:
 - `SmartBulb`
 - Properties:
 - `is_on`: is this bulb on?
 - `brightness`: the brightness of the bulb, values in [0,100]
 - `device_type`: will be "BULB"
 - `SmartThermostat`
 - Properties:
 - `current_temp`: current temperature
 - `target_temp`: the target temperature
 - `humidity`: the current humidity
 - `device_type`: will be THERMOSTAT
 - `SmartCamera`
 - Properties:
 - `motion_detected`: is motion detected?
 - `battery_level`: the level of the battery, values in [0,100]

- `last_snapshot`: the time when camera took the last snapshot
- `device_type`: will be CAMERA
- **Encapsulation**: Devices must protect their internal state (e.g., a bulb cannot have negative brightness).

2. The Network Simulation (AsyncIO)

- **Concurrency**: The central controller must listen to all devices "simultaneously" without blocking.
- **Event Loop**: Use `asyncio` to simulate a stream of incoming data packets. Each device should randomly "sleep" for a few seconds and then push a status update.
- **Task Management**: The controller must use `asyncio.create_task` or `TaskGroup` to manage hundreds of device connections effectively.

3. The Analytics Engine (Functional Programming)

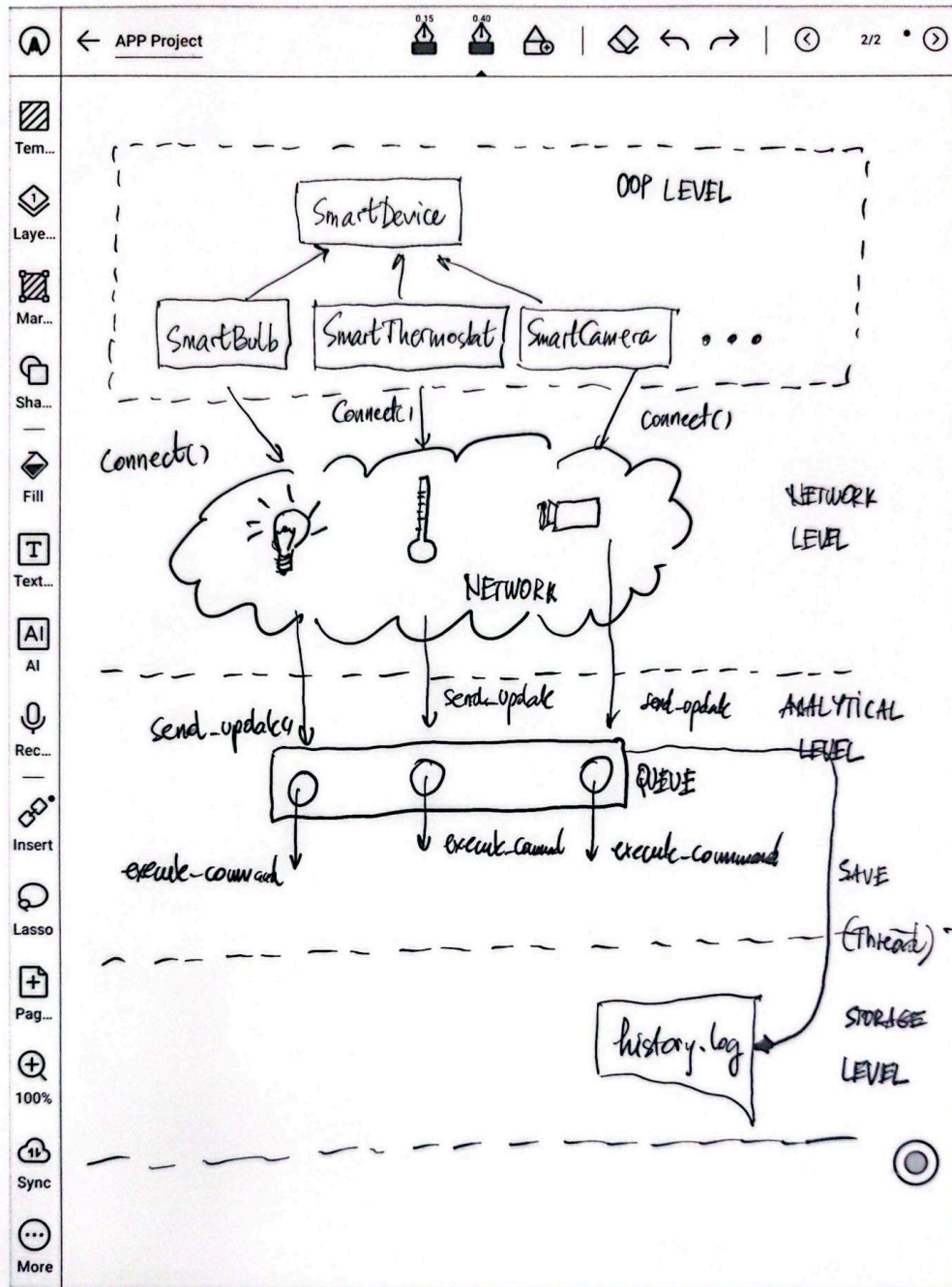
- **Pipeline Processing**: Incoming data must be processed using a functional pipeline, not imperative loops.
- **Map**: Transform raw JSON data into standard named tuples or Dataclasses (e.g., `(DeviceID, Timestamp, Value)`).
- **Filter**: Create a stream that filters for critical events (e.g., `temp > 30` or `battery < 10%`).
- **Reduce**: Calculate real-time metrics, such as "Total Energy Consumption" or "Average House Temperature" using `functools.reduce`.

4. The Storage System (Threading)

- **Non-Blocking I/O**: Writing device history to a log file (`history.log`) is slow (IO-bound/blocking).
 - **Threaded Worker**: The main async loop should dump processed data into a `queue.Queue`.
 - **Daemon Thread**: A separate thread must continuously pull from this queue and write to disk. This ensures that if the disk write hangs for 1 second, the Smart Lights don't stop responding during that second.
-

EcoHub High-Level Schema

BOOK



The program flow:

1. Define a thread that saves data to the history.log file. The thread looks into a Queue object for data
2. Instantiate a few smart devices
3. Connect them to the network (by connecting them to the network, the property `is_connected` becomes `True`)
4. In an infinite loop, make them send updates from time to time. The update consists of a dictionary with the properties and their values at that moment in time
5. You may filter from the raw updates only specific devices and execute commands for the selected ones (e.g., select only thermostats for which the current temperature is higher than a specified value)
6. Take the raw updates and put them in the Queue defined at point 1). In this way, the values are saved on the disk

Possible console output

```
Storage Thread Started...
Connecting devices...
Mysterious Smart Bulb is connecting...
Famous Smart Thermostat is connecting...
Gorgeous Smart Camera is connecting...
Gorgeous Smart Camera connected successfully in 0.94s.
Mysterious Smart Bulb connected successfully in 1.32s.
Famous Smart Thermostat connected successfully in 1.49s.
All devices connected!
....! ALERT: High Temp detected! Triggering cooling...
Smart Thermostat command executed: Temperature adjusted.
.....! ALERT: High Temp detected! Triggering cooling...
Smart Thermostat command executed: Temperature adjusted.
.! ALERT: High Temp detected! Triggering cooling...
Smart Thermostat command executed: Temperature adjusted.
.....! ALERT: High Temp detected! Triggering cooling...
Smart Thermostat command executed: Temperature adjusted.
```

.⚠️ ALERT: High Temp detected! Triggering cooling...
Smart Thermostat command executed: Temperature adjusted.

Extract from the history.log file

```
{'device_id': 'bulb_01', 'type': 'BULB', 'timestamp':  
1764752054.2872279, 'payload': {'is_on': False, 'brightness': 100}}
```

```
{'device_id': 'thermo_01', 'type': 'THERMOSTAT', 'timestamp':  
1764752054.2872388, 'payload': {'current_temp': 23.136089595323497,  
'target_temp': 24.0, 'humidity': 45.968590617178975}}
```

```
{'device_id': 'cam_01', 'type': 'CAMERA', 'timestamp':  
1764752054.287252, 'payload': {'motion_detected': False,  
'last_snapshot': 1764752054.287256, 'battery_level':  
59.98860539169797}}
```

```
{'device_id': 'bulb_01', 'type': 'BULB', 'timestamp':  
1764752056.389586, 'payload': {'is_on': False, 'brightness': 100}}
```

```
{'device_id': 'thermo_01', 'type': 'THERMOSTAT', 'timestamp':  
1764752056.3896, 'payload': {'current_temp': 19.34479331650711,  
'target_temp': 24.0, 'humidity': 34.30919764933631}}
```

```
{'device_id': 'cam_01', 'type': 'CAMERA', 'timestamp':  
1764752056.3896132, 'payload': {'motion_detected': False,  
'last_snapshot': 1764752056.389618, 'battery_level':  
27.503925104343914}}
```